PORTFOLIO

연초희

Java 백엔드로 B2B 시스템을 만들어 왔고, 최근에는 RAG 챗봇을 혼자 기획하고 주요 개발을 맡아 배포까지 마쳤습니다. 아래는 그 과정에서 무엇이 문제였고 어떻게 판단했는지를 프로젝트별로 정리한 것입니다.

duschgml0228@gmail.com
github.com/yeonchohee
bugcookie.tistory.com

한눈에 보기

프로젝트	기간	역할	핵심 기술
Workbee RAG 문서 챗봇	2025.07 ~ 현재	단독 기획, 주요 개발	Python, FastAPI, LangChain, ChromaDB
건설 현장 통합 대시보드	2026.06 ~ 현재	단독 풀스택 개발, 배포	Spring Boot, JPA, JavaScript, ACC/APS API, GitLab CI/CD
차량 IoT 관제 시스템(FMS)	2022.03 ~ 2023.09	웹·앱 개발	Spring Boot, Vue.js, Flutter, AWS
maxpolio	2026.09 ~ 현재	개인 프로젝트	Node.js, Claude Code Hooks
doc-rag-chat	2026.05	개인 프로젝트	FastAPI, React, ChromaDB, GitHub Actions

일하는 방식

- 만들기 전에 "이게 왜 필요한가"를 먼저 묻습니다. 1년간 개인 사업(스터디라운지 운영 등)을 하면서, 만드는 사람이 아니라 운영하는 사람의 입장에서 서비스를 보게 됐습니다.
- 도메인이 바뀌어도 빠르게 적응합니다. 법무, 제조, 차량 IoT, 건설을 거치며 Java에서 C#, Vue.js, Flutter, Python으로 필요한 기술을 그때그때 익혀 썼습니다.
- AI 코딩 도구를 일상적으로 씁니다. 요구사항과 계획을 로드맵 문서와 단계별 프롬프트로 관리하고, 결과는 직접 검증합니다.

01 Workbee

RAG 기반 문서 Q&A 챗봇

단독 기획, 주요 개발 | **2025.07 ~ 현재**

문제	건축사 사무소의 대량 문서에서 필요한 내용을 찾기 어렵다는 고객 요청이 반복됨
한 일	RAG 챗봇을 직접 제안하고 인덱서, 검색 API, 임베드형 채팅 위젯을 설계·개발
결과	사내 테스트 운영 중. 자사 서비스 적용 준비 단계

Python | FastAPI | LangChain | ChromaDB | bge-m3 | EXAONE | Docker

고객사의 반복 요청에서 출발해 직접 제안하고 혼자 기획하고 주요 개발을 맡은 프로젝트입니다. RAG와 LLM을 먼저 공부해 팀에 공유했고, 프롬프트 작성은 팀장에게 부탁해 나눠 맡았습니다. (주)상상진화, 2025.07 ~ 현재.

문제

건축사 사무소는 프로젝트마다 문서가 대량으로 쌓입니다. 고객사에서 "문서 찾기가 어렵다"는 요청이 반복됐습니다.

판단

- 문서가 워낙 방대해 원하는 문서를 찾기 힘들다는 요청이었기 때문에, 문서를 직접 찾지 않고 자연어로 묻고 답을 받는 RAG 챗봇을 처음부터 제안했습니다.
- 문서가 계속 바뀌는 환경이라, 매번 전체를 다시 임베딩하지 않는 구조가 필요했습니다.
- 고객 문서가 외부로 나가지 않게 하고 사용료도 줄이기 위해, 외부 API 대신 내부 서버에서 돌리는 sLLM을 쓰기로 했습니다. 모델은 바꿀 수 있게 만들어 두고, 테스트 단계에서는 무료이고 한국어 성능이 좋은 EXAONE을 쓰고 있습니다.

기획

- 제안서를 두 차례, 기획안을 세 차례 고쳐 쓰며 방향을 구체화했습니다.
- 텍스트 문서 검색부터 시작해 단계별로 넓혀 가는 MVP 우선 로드맵을 세웠습니다.
- 팀의 Java 경험을 바탕으로 하되, AI 생태계가 풍부한 Python을 택하고 학습 기간을 로드맵 첫 단계에 넣었습니다.
- 고객의 원본 데이터는 외부로 내보내지 않고 내부 서버에서만 처리한다는 원칙을 처음부터 정했습니다.

준비

RAG를 다뤄 본 적이 없는 상태에서 시작했습니다. 그래서 학습을 따로 떼지 않고 프로젝트 일정표(WBS)의 첫 단계로 넣었습니다. 주제마다 목표와 할 일을 정했고, 공부한 내용은 노션에 기록해 팀과 공유했습니다.

시기	단계	목표
2025.07 ~	기획	제안서와 기획안 작성
2025.09 1주	AI 기본 개념, 개발 환경	LLM과 RAG가 무엇인지 이해
2025.09 2~3주	벡터 DB 학습, 아키텍처 분석	RAG 문서 챗봇 구축 사례 2~3개를 분석해 구성요소의 역할 이해
2025.09 3~4주	파이썬과 API 기초	실제 개발에 필요한 파이썬 기본기 확보
2025.09 4주	문서 전처리 실습	문서 로더와 텍스트 분할로 문서를 검색에 맞게 가공
2025.09 말 ~ 10 초	벡터 DB 구축	학습한 내용으로 실제 벡터 DB 구성
2025.10 중 ~ 11 중	핵심 기능 개발	검색과 답변 기능 개발, 코드 리뷰 2회
2025.11 중 ~ 12 초	테스트와 개선	답변 품질 점검과 보완

구성

구성	하는 일
인덱서	문서를 읽어 전처리·임베딩하고 벡터 DB에 저장. 변경·삭제된 파일만 반영
검색 API	질문을 받아 벡터 검색을 하고, 출처 번호를 붙여 답변 생성
채팅 위젯	다른 웹 사이트에 iframe으로 붙여 쓰는 플로팅 채팅창

한 일

1. 문서 수집 → 임베딩 → 검색 → 답변 생성의 파이프라인을 LangChain과 ChromaDB로 설계·구현 (PDF·DOCX, 이미지 PDF는 OCR)
2. 바뀐 파일만 다시 임베딩하는 증분 인덱싱 적용
3. 검색된 문서에 출처 번호를 붙여 답변에서 [1], [2]로 인용하는 구조 설계. 프롬프트 작성은 팀장이 맡음
4. 한 서버로 여러 챗봇을 제공하는 임베드형 채팅 위젯과 스트리밍 응답 구현
5. 모델을 환경변수로 교체할 수 있게 하고 내부 서버 sLLM(EXAONE)으로 운영, 테스트 132개를 갖춰 Docker로 배포

결과

혼자 기획하고 주요 개발을 맡아 배포까지 마쳤습니다. 현재 사내 테스트용으로 운영 중이며, 자사 서비스에 적용하는 것을 준비하고 있습니다. 문서 Q&A에서 추천 기능으로 확장하는 후속 설계도 진행 중입니다.

02 건설 현장 통합 대시보드

단독 풀스택 개발, 외부 연동, 배포 | **2026.06 ~ 현재**

문제	PM이 Claude Code로 만든 화면 목업만 있고 실제 데이터와 백엔드가 없었음. 여러 현장을 다룰 수 없는 구조, 수동 배포
한 일	목업 화면을 그대로 구현하고 실제 데이터를 연결해 단독으로 풀스택 개발. ACC 대용량 문서를 이미지로 캐시해 표시, 멀티 현장 구조 전환, GitLab CI/CD 구축
결과	임원진에게 좋은 반응, 후속 시스템 고도화 2건 논의. API 105개 · 테스트 770개

Java 17 · Spring Boot 3 · JPA · MariaDB · JavaScript(ES 모듈) · Autodesk ACC/APS API · GitLab CI/CD

PM이 Claude Code로 만든 HTML 목업 화면을 그대로 구현하고, 실제 데이터를 연결해 서비스로 만든 프로젝트입니다. Autodesk ACC 문서, BIM 모델, 현장 공사 정보를 현장 단위로 한 화면에 모으는 Spring Boot 기반 웹 대시보드로, 관리자 페이지를 제외한 본체를 화면부터 백엔드 API, 외부 연동, 배포까지 혼자 풀스택으로 개발했습니다. API 105개, 테스트 770개 규모입니다. (주)상상진화, 2026.06 ~ 현재.

기획자가 만든 목업 화면을 그대로, 실제 데이터로

- **문제** PM이 Claude Code로 만든 HTML/CSS/JS 목업이 출발점이었습니다. 화면은 완성돼 있었지만 데이터는 모두 더미였고, 화면 코드는 2,005줄짜리 스크립트 하나였습니다.
- **판단** 화면을 새로 만들지 않고 목업을 그대로 살리기로 했습니다. 화면 결과물, 색상, 위젯 id는 바꾸지 않는다는 원칙을 먼저 정했습니다.
- **한 일** 목업을 분석해 위젯별로 필요한 API와 데이터 구조를 설계 문서로 정리하고, 목업의 더미 데이터 계층만 실제 API 호출로 교체했습니다.
- **한 일** 단일 스크립트를 번들러 없이 브라우저 네이티브 ES 모듈로 위젯 단위 분리했습니다. 단계마다 실제로 띄워 화면과 주요 기능을 확인한 뒤 다음 단계로 넘어갔습니다.
- **결과** 임원진(대표, 전무, 영업 담당)에게 좋은 반응을 얻었고, 이를 계기로 후속 시스템 고도화 2건이 논의되고 있습니다.

ACC 대용량 문서를 이미지로 저장해 바로 보여주기

- **문제** 대시보드에 보여줄 문서는 ACC에 있는데, 도면·PDF처럼 용량이 큰 파일이 많았습니다. 대시보드를 열 때마다 원본을 불러오면 시간이 너무 오래 걸렸습니다.
- **한 일** 위젯마다 보여줄 문서와 페이지를 골라 "대시보드에 저장"하면 DB에 기록하고, 서버가 그 페이지를 이미지(썸네일)로 만들어 디스크에 캐시하게 했습니다. 대시보드는 원본 대신 이 이미지를 보여줍니다.
- **한 일** 원본은 파일 버전 기준으로 캐시해, 같은 버전이면 ACC에서 다시 받지 않고 새 버전이 올라왔을 때만 새로 받게 했습니다. 이미 캐시된 이미지는 ACC 인증을 거치지 않고 바로 응답합니다.
- **결과** 큰 파일도 대시보드를 열 때 원본을 받지 않고 저장된 이미지로 바로 표시됩니다. 보여줄 페이지도 DB에 남아 있어, 다른 브라우저나 기기에서 열어도 같은 화면이 나옵니다.

배포 파이프라인이 없던 환경에 CI/CD 구축

- **문제** 온프레미스 서버에 수동으로 배포하고 있었고, 자동 배포 환경이 없었습니다.
- **판단** 자동화부터 하지 않고 수동 배포 절차를 먼저 검증한 뒤, 그 절차를 그대로 파이프라인으로 옮겼습니다.
- **한 일** Self-hosted Runner를 직접 설치하고, 빌드·배포 스크립트를 공통 템플릿으로 둔 채 브랜치와 Runner 태그로 개발·운영을 분리했습니다.
- **보안** 배포 계정에는 배포에 필요한 최소 권한만 부여했습니다.
- **결과** 개발 서버에 이어 운영 서버의 첫 자동 배포까지 완료했습니다.

설정 파일 고정값을 멀티 프로젝트 구조로 전환

- 문제 허브·프로젝트·폴더 값이 설정 파일에 고정되어 있어 현장 하나만 다룰 수 있었습니다.
- 한 일 현장·사용자·폴더 정보를 DB 테이블로 매핑하고, 접근 가능한 현장 조회 API와 세션 기반 현재 현장 선택을 구현했습니다.
- 결과 대시보드 레이아웃, 폴더 위젯, 메뉴 노출이 선택한 현장 기준으로 동작합니다. 첫 화면에서 위젯별로 나가던 API 요청 15건(위젯 이름 10건, 슬롯 문서 5건)은 배치 조회 2건으로 묶었습니다.

03 차량 IoT 관제 시스템(FMS)

웹·앱 개발 | 2022.03 ~ 2023.09

문제	차량의 위치와 상태를 실시간으로 보고, 웹·앱에서 실제 차량을 제어해야 함
한 일	단말 데이터 파싱·적재, 실차 원격 제어, 경로 시각화, Vue.js 도입, Flutter 하이브리드 앱 전환
결과	단말이 BLE 동글에서 퀄컴 칩으로 바뀔 때 같은 데이터 구조로 이관

Java | Spring Boot | Vue.js | Flutter | Python | AWS

차량에서 올라오는 데이터를 받아 위치와 상태를 실시간으로 보여주고, 웹·앱에서 실제 차량을 제어하는 시스템입니다. 주식회사 드림에이스 클라우드팀, 2022.03 ~ 2023.09.

데이터 흐름

1. 차량 단말(테스트용 BLE 동글, 이후 차량 내장 퀄컴 칩)이 센서·GPS 데이터를 전송
2. AWS의 Python 서비스가 원시 데이터를 파싱·정형화
3. API 서버가 가공된 데이터를 앱과 관리자 웹에 제공
4. 운전자는 앱에서 차량 상태 확인과 제어, 관리자는 웹에서 위치·경로 모니터링

한 일

- BLE 테스트 장비의 원시 데이터를 분석해 JSON 구조로 파싱하고, 서버에 자동 적재하는 로직을 직접 개발. 단말이 퀄컴 칩으로 바뀔 때 같은 구조로 연동되도록 이관
- 문 잠금/해제, 경적, 라이트를 웹·앱에서 제어하는 기능 구현
- GPS 데이터를 지도 위 실시간 경로로 그리는 관리자 화면 개발
- 백엔드 하나로 관리자 웹과 운전자 앱을 함께 서비스하도록, Thymeleaf 서버 렌더링 화면을 프론트엔드·백엔드 분리 구조로 바꾸자고 직접 제안해 채택되었고, 팀에 Vue.js를 도입해 REST API + Vue.js 구조로 마이그레이션
- 외주로 관리가 어렵던 운전자 앱을, 팀이 익힌 Vue.js 웹 UI를 활용해 Flutter + WebView 하이브리드 앱으로 자체 개발. 앱 배포와 유지보수를 단순화
- AWS Cognito 인증·권한 분리, S3 첨부파일, 배치 기반 알림 메일 발송 구현

배운 것

하드웨어가 바뀌면 데이터 구조도 바뀝니다. BLE 동글에서 만든 파싱 구조를 퀄컴 칩 기반 차량에도 그대로 쓸 수 있게 맞춰 이관했습니다.

04 maxpolio

Claude Code 세션 자동 저널 도구

개인 프로젝트 | 2026.09 ~ 현재

문제	직접 쓰는 작업 기록은 오래가지 않아 포트폴리오가 쌓이지 않음
한 일	세션 종료 훅과 작업 스케줄러로 세션 기록을 자동으로 저널화하는 도구 제작
결과	밀려 있던 세션 7개 복구. 이 포트폴리오의 최근 내용이 이 도구의 저널에서 나옴

Node.js | Claude Code Hooks | Windows 작업 스케줄러

개발 세션 기록과 git 이력을 분석해 기술 저널과 포트폴리오 자료(강점 리포트)를 자동으로 쌓는 개인 도구입니다. 이 포트폴리오의 최근 프로젝트 내용도 이 도구가 쌓은 저널에서 나왔습니다. 시연 보기 · GitHub, 2026.09 ~ 현재.

문제

매일 한 일을 기록해야 포트폴리오가 쌓이는데, 직접 쓰는 기록은 오래가지 않았습니다. 개발 대부분을 Claude Code로 하고 있으니, 세션 기록 자체가 재료가 될 수 있다고 봤습니다.

구조

1. 세션 종료 훅이 세션 정보를 큐 파일로 기록
2. 작업 스케줄러가 주기적으로 큐와 열려 있는 세션을 확인
3. 세션 기록 파싱 → git 이력 대조 → 분류와 글 생성 → 마크다운 저장
4. 누적된 저널에서 강점 리포트와 채용 사이트용 이력서 초안 생성

디버깅 사례: "왜 다른 프로젝트에서는 기록이 안 쌓이지?"

- **증상** 훅은 정상 등록돼 있는데 3주 동안 저널이 거의 쌓이지 않았습니다.
- **원인 1** 생성 프로세스가 종료 코드 129(SIGHUP)로 죽고 있었습니다. 성공한 실행은 31~36초가 걸렸는데, 실패한 실행은 세션 종료 4~27초 뒤에 끝났습니다. 터미널을 닫을 때 분리 실행한 워커까지 함께 정리되는 것이었습니다.
- **원인 2** 세션을 며칠씩 열어 두기 때문에 종료 훅 자체가 거의 불리지 않았습니다.
- **해결** 훅은 큐 기록만 하고, 생성은 세션과 무관한 작업 스케줄러가 처리하도록 분리했습니다. 열려 있는 세션도 2시간 이상 조용하면 기록합니다.
- **함께 발견** Node 24의 파일 삭제 함수가 한글 경로에서 예외 없이 프로세스를 죽이는 문제를 재현해 우회했습니다.
- **결과** 밀려 있던 세션 7개가 모두 저널로 복구됐습니다.

신뢰성을 위한 제약

- 세션 로그와 diff에 없는 내용은 쓰지 않도록 프롬프트에서 제한
- 로그의 토큰·비밀번호를 생성 전에 마스킹
- 소스 저장소와 저널 저장소를 분리해 회사 내부 내용이 공개 저장소에 섞이지 않게 구성

05 doc-rag-chat

누구나 써 볼 수 있는 RAG 공개 데모

개인 프로젝트 2026.05

한 일 문서 업로드, 세션별 대화 기록, SSE 스트리밍, 관리자 대시보드, CI/CD 자동 배포를 혼자 설계·배포

FastAPI ChromaDB GPT-4o-mini React SQLite GitHub Actions

회사 RAG 챗봇은 코드도 화면도 공개할 수 없어서, 같은 경험을 누구나 직접 써 볼 수 있는 공개 데모로 만들었습니다. 코드와 라이브 데모를 모두 공개했습니다. 라이브 데모, 2026.05.

- 사용자가 문서를 올리면 바로 청킹, 임베딩, 벡터 DB 저장까지 처리
- 접속자(세션)마다 벡터 컬렉션을 따로 두어 데이터를 격리하고, 세션을 지우면 한 번에 삭제
- SSE 스트리밍 응답과 답변 출처 표시, 관리자 화면
- GitHub Actions로 테스트 통과 후 백엔드(Render)와 프론트엔드(GitHub Pages) 자동 배포
- 청크 크기(500자, overlap 50)와 임베딩 모델 선택 근거를 README에 정리
- FastAPI, LangChain, ChromaDB, GPT-4o-mini, React, SQLite